Asymptotic Analysis and Formal Implementations of Singly-Linked List Operations: Midpoint Detection, Sorted Merging, Merge Sort, and Palindrome Verification

Abstract

This paper presents a formal analysis and reference implementations of key singly-linked list algorithms. We study the problem of midpoint detection using Floyds cycle-finding pointer mechanics (fast and slow pointers), proving its efficiency over naive two-pass techniques. We then present an optimal comparison-based sorted list merging algorithm using dummy nodes to simplify pointer re-linking. Building on these two primitives, we formalize the Merge Sort paradigm on singly-linked lists, analyzing its $O(N \log N)$ worst-case time complexity and $O(\log N)$ auxiliary space complexity. Finally, we address the problem of palindrome verification in singly-linked lists, contrasting the naive $O(N)$ space list cloning approach with an optimal $O(1)$ auxiliary space algorithm based on mid-point list reversal and restoration. Complete Java and Python implementations are provided with execution traces, complexity proofs, and TikZ illustrations.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Linked lists are foundational linear data structures in computer science, characterized by nodes that are dynamically allocated in memory and linked via pointer references. Unlike arrays, which occupy contiguous memory blocks and support constant-time random access, linked lists store elements non-contiguously. This design offers distinct advantages, such as $O(1)$ insertions and deletions, but introduces challenges, particularly the necessity of linear-time sequential traversal to access elements.

In algorithmic design and competitive programming, managing linked list structures efficiently is a critical skill. This paper focuses on four core problems:

- Midpoint Detection: Identifying the center of a list in a single pass.
- Sorted Merging: Combining two sorted lists into a single sorted list.
- Merge Sort: Sorting a list in $O(N \log N)$ time.
- Palindrome Verification: Validating if a list reads the same forwards and backwards.

We present a rigorous treatment of these problems, demonstrating naive and optimal strategies, analyzing their computational complexities, and detailing their implementations in Java and Python.

2 Midpoint Detection

Finding the middle node of a linked list is a fundamental operation, often serving as a preprocessing step for divide-and-conquer algorithms like Merge Sort.

2.1 Problem Statement and Naive Approach

Given a singly-linked list, the objective is to find the node at index $\lfloor N/2 \rfloor$ (0-based indexing), where N is the total number of nodes. If the list is empty, return null.

The naive approach involves a two-pass strategy:

1) Traverse the list from the head to count the total number of nodes, N.
2) Traverse the list again from the head for $\lfloor N/2 \rfloor$ steps to reach and return the middle node.

Although the time complexity is asymptotically linear, $O(N)$, it requires approximately $1.5N$ node accesses (a full pass followed by a half pass). The space complexity is $O(1)$ as only a count variable and a temporary pointer are required.

2.2 Optimal Fast-Slow Pointer Approach

An optimal, single-pass strategy employs Floyds cycle-finding pointer mechanics, commonly referred to as the Tortoise and Hare algorithm. Two pointers, slow and fast, are initialized to the head of the list. In each step, the slow pointer advances by one node, while the fast pointer advances by two nodes.

Theorem 1. When the fast pointer reaches the end of the list (i.e., fast is null or fast.next is null), the slow pointer will reside exactly at the midpoint node.

Proof. Let N be the number of nodes in the list. The fast pointer moves at twice the speed of the slow pointer. Therefore, if the fast pointer covers a distance of d nodes, the slow pointer covers a distance of $d/2$ nodes. When the fast pointer reaches the end of the list after traversing approximately N nodes, the slow pointer has traversed $N/2$ nodes, which corresponds to the middle node. □

This approach completes the operation in a single pass ($O(N)$ time with N node accesses) and uses $O(1)$ auxiliary space.

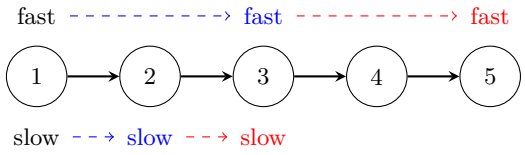

Fig. 1. Progression of slow and fast pointers on a list of length 5. At step 0, both are at node 1. At step 1 (blue), slow is at 2, fast is at 3. At step 2 (red), slow is at 3 (the middle), and fast is at 5 (end of list, since fast.next is null).

3 Merging Two Sorted Linked Lists

Given two sorted singly-linked lists, A and B, the objective is to merge them into a single sorted list, C. The merge must be performed in-place by adjusting node pointers, without allocating new nodes.

3.1 Dummy Node Technique

A classic software engineering pattern to simplify linked list manipulations is the use of a dummy head (or placeholder node). Without a dummy node, the algorithm must handle the initialization of the merged list's head pointer as a special case. With a dummy node, the head is anchored, and all re-linking operations follow the same uniform logic.

The algorithm proceeds as follows:

1) Create a dummy node and initialize a traversal pointer, temp, to point to it.
2) Maintain pointers t1 and t2 starting at the heads of lists A and B respectively.
3) Compare the data values of t1 and t2. Link temp.next to the node with the smaller value, and advance that list's pointer.
4) Advance temp.
5) Repeat steps 3 and 4 until one of the lists is exhausted.
6) Attach the remaining nodes of the non-exhausted list directly to temp.next.
7) Return dummy.next as the head of the merged list.

3.2 Complexity Analysis

The time complexity of this operation is $O(N + M)$, where N and M are the lengths of lists A and B respectively. This is because every node from both lists is visited and linked exactly once. The auxiliary space complexity is $O(1)$, as we only allocate a few pointer references and a single dummy node.

4 Merge Sort on Singly-Linked Lists

Sorting is a fundamental operation. While Quick Sort is often preferred for contiguous arrays due to cache locality, Merge Sort is the algorithm of choice for linked lists.

4.1 Why Merge Sort?

Quick Sort relies on random access to select pivots and partition arrays efficiently. Since linked lists do not support constant-time random access, partitioning becomes inefficient. Conversely, Merge Sort is naturally suited for linked lists because:

- It divides lists at the midpoint, which can be done in $O(N)$ time.
- Merging two sorted linked lists requires only $O(1)$ auxiliary space, overcoming Merge Sort's main drawback on arrays (which require $O(N)$ temporary storage).

4.2 Algorithm and Trace

The Merge Sort algorithm on linked lists uses a divide-and-conquer strategy:

1) Divide: Find the middle node of the list. Split the list into two halves by setting the next pointer of the node preceding the middle node to null.
2) Conquer: Recursively apply Merge Sort to both the left and right sublists.
3) Combine: Merge the two sorted sublists using the sorted merge routine.

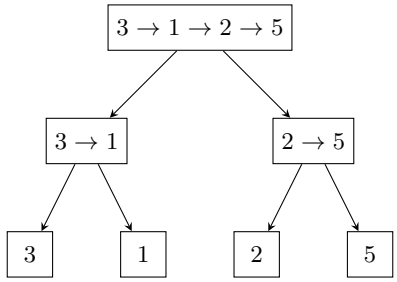

Fig. 2. Divide phase: splitting the list $3 \to 1 \to 2 \to 5$ down to single-element base cases.

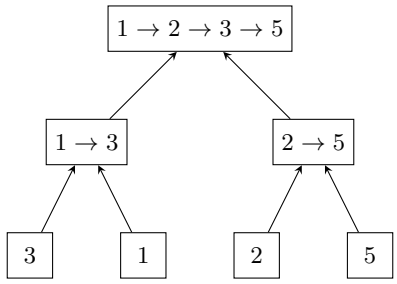

Fig. 3. Conquer and Combine phase: merging sorted sublists back to the final sorted list.

4.3 Complexity Analysis

The recurrence relation for Merge Sort is:

$$T(N) = 2T(N/2) + O(N) \tag{1}$$

By the Master Theorem, this yields a time complexity of $O(N \log N)$ for best, worst, and average cases. The space complexity is $O(\log N)$ due to the recursive call stack, which has a maximum depth proportional to the height of the recursion tree, $\log_2 N$.

5 Palindrome Verification

A list is a palindrome if its elements read the same in forward and backward directions. We compare the naive and optimal strategies.

5.1 Naive Approach

The naive approach clones the original list, reverses the cloned list, and then performs a simultaneous node-by-node value comparison. While simple, this strategy has a space complexity of $O(N)$ to store the cloned list, making it sub-optimal for systems with constrained memory.

5.2 Optimal Midpoint Reversal Approach

An optimal algorithm checks the palindrome property in-place, achieving $O(1)$ auxiliary space. The steps are:

1) Find the middle of the list using the fast-slow pointer technique.
2) Reverse the second half of the list in-place.
3) Compare the values of the first half (starting from the original head) with the reversed second half.
4) Restore the list to its original state by reversing the second half again and reconnecting it to the first half.
5) Return the comparison result.

Theorem 2. The optimal palindrome check runs in $O(N)$ time and $O(1)$ space, and leaves the list structure unmodified.

Proof. Let N be the number of nodes. Midpoint detection takes $O(N)$ time. Reversing the second half (length $N/2$) takes $O(N)$ time. Comparing the two halves takes $O(N)$ time. Restoring the second half takes $O(N)$ time. The total time is $O(N)$. Since all operations are performed by mutating pointer variables in-place, the auxiliary space complexity is $O(1)$. □

6 Implementations

We present full, clean implementations of the optimal algorithms in Java and Python.

6.1 Java Implementation

```
public class LinkedListAlgorithms {
    public static class ListNode {
        int val;
        ListNode next;
        ListNode(int val) { this.val = val; }
    }

    // Q1: Find Middle Node
    public ListNode findMiddle(ListNode head) {
        if (head == null) return null;
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    // Q2: Merge Two Sorted Lists
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(-1);
        ListNode current = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                current.next = l1;
                l1 = l1.next;
            } else {
                current.next = l2;
                l2 = l2.next;
            }
            current = current.next;
        }
        if (l1 != null) current.next = l1;
        if (l2 != null) current.next = l2;
        return dummy.next;
    }

    // Q3: Merge Sort
    public ListNode mergeSort(ListNode head) {
        if (head == null || head.next == null) return head;

        // Find split point
        ListNode prev = null;
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }
        prev.next = null; // Split the list

        ListNode l1 = mergeSort(head);
        ListNode l2 = mergeSort(slow);
        return mergeTwoLists(l1, l2);
```

```
    }

    // Q4: Palindrome Check (with restoration)
    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;

        // 1. Find middle node
        ListNode prev = null;
        ListNode slow = head;
        ListNode fast = head;
        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }

        // 2. Reverse second half
        ListNode secondHalfHead = reverse(slow);

        // 3. Compare values
        ListNode p1 = head;
        ListNode p2 = secondHalfHead;
        boolean result = true;
        while (p2 != null) {
            if (p1.val != p2.val) {
                result = false;
                break;
            }
            p1 = p1.next;
            p2 = p2.next;
        }

        // 4. Restore list
        reverse(secondHalfHead);

        return result;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;
        while (curr != null) {
            ListNode nextNode = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextNode;
        }
        return prev;
    }
}
```

6.2 Python Implementation

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class LinkedListAlgorithms:
    # Q1: Find Middle
    def findMiddle(self, head: ListNode) -> ListNode:
```

```
    if not head:
        return None
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow

# Q2: Merge Sorted Lists
def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
    dummy = ListNode(-1)
    current = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            current.next = l1
            l1 = l1.next
        else:
            current.next = l2
            l2 = l2.next
        current = current.next
    if l1:
        current.next = l1
    if l2:
        current.next = l2
    return dummy.next

# Q3: Merge Sort
def mergeSort(self, head: ListNode) -> ListNode:
    if not head or not head.next:
        return head

    prev = None
    slow = fast = head
    while fast and fast.next:
        prev = slow
        slow = slow.next
        fast = fast.next.next
    prev.next = None  # Split

    l1 = self.mergeSort(head)
    l2 = self.mergeSort(slow)
    return self.mergeTwoLists(l1, l2)

# Q4: Palindrome Check
def isPalindrome(self, head: ListNode) -> bool:
    if not head or not head.next:
        return True

    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    second_half = self._reverse(slow)

    p1, p2 = head, second_half
    result = True
    while p2:
        if p1.val != p2.val:
            result = False
            break
```

```
            p1 = p1.next
            p2 = p2.next

        self._reverse(second_half)  # Restore
        return result

    def _reverse(self, head: ListNode) -> ListNode:
        prev = None
        curr = head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        return prev
```

7 Complexity Summary

TABLE 1
Complexity Analysis of Core Singly-Linked List Algorithms.

Algorithm	Time Complexity	Auxiliary Space	Key Concept
Midpoint Detection	$O(N)$	$O(1)$	Two-pointer (Fast/Slow)
Sorted Merging	$O(N+M)$	$O(1)$	Dummy head anchoring
Merge Sort	$O(N \log N)$	$O(\log N)$	Divide-and-conquer recursion
Palindrome Check (Naive)	$O(N)$	$O(N)$	Full cloning and reversal
Palindrome Check (Optimal)	$O(N)$	$O(1)$	In-place reversal and restoration

8 Conclusion

This paper explored fundamental algorithms for singly-linked lists, demonstrating that pointer-manipulation techniques can optimize both execution efficiency and memory footprints. Floyds fast-slow pointer algorithm enables midpoint detection in a single pass. By combining midpoint split logic with comparisons and dummy-node anchoring, Merge Sort is realized in optimal $O(N \log N)$ time and $O(\log N)$ stack space. Finally, we demonstrated that in-place pointer mutations allow verification of the palindrome property in $O(N)$ time and $O(1)$ auxiliary space while preserving the structural integrity of the input list. These primitives serve as the building blocks for constructing complex memory-efficient systems and solving advanced data structure problems.